SeniorPadi: Technical Report

Capstone project. Repository: github.com/Otani-ibe/SeniorPadi_Capstone. Walkthrough video and Figma designs are linked in the README.

SeniorPadi is a mobile-first website for people aged 50 and over in Ibadan, Nigeria. Seniors find community events, register or volunteer, and meet other verified seniors. Churches, NGOs and community groups run the events from an admin area. It is a server-rendered Node.js app (Express 5, EJS, Tailwind CSS) on PostgreSQL through Sequelize, with Socket.IO and WebRTC for chat and calls. The code is about 14,000 lines of JavaScript and templates, with 26 database migrations and 32 test suites.

1. Scope alignment and feature validation

The table compares the project with the scope in `docs/MVP.md`. Four things changed from the first plan, and I explain them under the table.

Planned feature	Purpose	Status	Evidence
Find events, one card at a time	Seniors with cheap phones see one clear choice: skip or register	Completed	`views/events/index.ejs`, `src/routes/events.js`, `tests/deck.test.js`
Register and volunteer, with limits on places	Fair places, and volunteers for each event	Completed	`src/services/capacity.js` (row lock in a transaction), `tests/capacity.test.js`
Sign up and log in by phone	Seniors who forget passwords can still get in	Completed, changed	Now a texted code, no password. `src/services/verification.js`, `tests/auth.test.js`
English and Yorùbá	Language should not block anyone	Completed, one gap	`locales/en.json`, `locales/yo.json`. A native speaker still needs to check the Yorùbá wording
SMS and email confirmations, reminders	Seniors remember to go	Partial	`src/services/notify.js`, `cron.js`. Works in code and tests. Live sending needs Brevo SMS credit, which I do not have yet, so demo mode shows codes on screen
Identity check before Find seniors	Keeps strangers out of the senior directory	Completed, changed	ID photo upload replaced by a NIN checked by the superadmin. `src/services/identity.js`, `tests/identity.test.js` (51 tests)
Find seniors, chat, online status, phone swap	Safe way to meet people	Completed	`src/routes/social.js`, `public/js/chat.js`, `tests/chat.test.js`, `social.test.js`
Voice calls, missed calls, call length	Talk without sharing a number	Completed	`src/sockets/calls.js`, `public/js/call.js`, `tests/calls.test.js`. The signalling is tested. The audio itself is not covered by automated tests
Report and block	Safety	Completed	`src/models/report.js`, `block.js`, `views/admin/reports.ejs`
Staff area: events, organisations, attendance, QR check-in, reports, staff accounts, dashboard	Run events without a technical team	Completed	`src/routes/admin.js`, `host.js`, `tests/admin.test.js`, `host.test.js`
Parent can sign up and book for a parent	Relatives often help online	Completed	`src/routes/bookForParent.js`, `tests/parent.test.js`
Nearby and recommended events (added)	Show the most useful events first	Completed	`src/services/recommendationScoring.js`, `nearby.js`, `geo.js`, matching tests
Tell seniors when an event changes or is cancelled (added)	Nobody turns up to the wrong time or place	Completed	`src/services/eventUpdates.js`, `tests/eventUpdates.test.js`, `cancelEvent.test.js`

Paying for paid events	Charge for some events	Not implemented	Price is shown on the event, but no payment is taken. Out of scope for the first version
Live deployment	Real use	Partial	`render.yaml` and a Neon database are set up. The live environment values are still being finished

Scope changes and why

- ID photo became a NIN check. Photos of IDs are risky to store. A senior now types their 11-digit NIN, it is saved encrypted, only the superadmin can read it, and it is deleted as soon as a decision is made. I first planned to use an outside identity service, but these need a registered business, so a human checks the NIN on the official NIMC website.
- Password login became a texted code. It is simpler for the target users. Staff still sign in with a password.
- Plain latitude and longitude columns instead of PostGIS, so the same code runs on a normal local database, the test database and the hosted one.
- Recommendations and event-change alerts were added because the first demos showed seniors still missed events near them and were confused when plans changed.

2. Technical implementation

A. Core functions and the logic behind them

- No overbooking. Registering opens a transaction and locks the event row (`SELECT ... FOR UPDATE`) before counting places, so two people cannot take the last one. Cancelling a place frees it for the next person.
- Recommendations. Each event gets a score out of 100: interest in the type 40, distance 30, past attendance 15, how soon 10, variety 5. The weights live in `src/config/recommendation.js`. Variety uses a rolling window so the deck does not show four fitness events in a row. Each card carries a short reason such as "Near you", built from the same numbers.
- Distance. A haversine function in `geo.js` gives kilometres between two points. A new address is looked up once (OpenStreetMap Nominatim) and saved, with the local government area centre as a fallback, so lookups never block a page.
- NIN security. AES-256-GCM encryption with keys derived by HKDF, the user id as authenticated data, and a separate HMAC hash so one NIN cannot open two accounts. A database CHECK constraint makes sure a NIN can only exist while a review is waiting.
- Chat and calls. Socket.IO carries messages and WebRTC call signalling, with a Cloudflare TURN relay for phones behind strict networks. Missed calls, answered calls and their length are saved as messages, so both people see them live.
- Event change alerts. The old and new event are compared to the minute. Staff choose "Save and notify registered seniors" or "Save without notifying", and the message lists exactly what changed.
- Front end. Pages are rendered on the server, so they work with JavaScript off. Small scripts add live chat, call buttons, the NIN box (digits only, 11 maximum) and the phone box.

B. Front end, back end and data handling

The browser sends forms and Socket.IO events to Express routes. Routes only handle the request and call a service. Services hold the rules and talk to Sequelize models, which map to PostgreSQL tables. EJS views turn the result into HTML. Every form carries a CSRF token, logins and costly actions are rate limited, and staff passwords are hashed with bcrypt.

C. Different inputs and error handling

Input	What the system does
NIN with letters, spaces, or more than 11 digits	The box removes non-digits and stops at 11. The server also rejects anything that is not exactly 11 digits
A NIN already used on another account	Refused, with a plain message that does not reveal whose it is
Wrong login code	Nobody is signed in and a clear message is shown. Repeated tries return 429 (too many attempts)
Registering for a full event, or after the deadline	No place is taken, and the senior is told why
A senior opening an admin page, or an admin opening the NIN page	403. Only the superadmin sees NINs
A form sent without a valid CSRF token	Rejected

Calling someone who is offline	A missed call is saved and shown to both people at once
Cancelling an event twice	Second request does nothing, so nobody is texted twice

D. Testing strategies

Result of the last full run: **32 test suites and 358 tests, all passing**.

- Integration testing (Jest and Supertest). I chose it because most of the risk is in how routes, rules and the database work together. Each test file resets a real PostgreSQL test database, signs in as a senior or admin, and sends real requests. It covers sign-up, registration limits, chat, calls, admin actions and permissions. When a test failed, I fixed the code: for example, a missed call only appeared once even after three attempts, so every attempt is now saved and sent live; and event edits lost the seconds of a time, so changes are compared to the minute.
- Unit testing. I chose it for the pure logic where a wrong number is hard to spot by eye: the scoring weights, diversity window, distance, NIN encryption (wrong key, moved to another account, tampered data), and date comparison. The tests check expected scores, ordering and edge cases such as missing coordinates.
- Usability and device checks. Because users have small screens and poor eyesight, I checked pages by hand and with a headless Chrome script at 390, 820 and 1280 pixels wide. I looked at whether every flyer shows whole, whether text covers an image, and whether the page scrolls sideways. Problems found: text covering a picture, a NIN box accepting extra digits, and a button style that looked inconsistent. All were fixed and the checks were repeated.

E. Performance and compatibility

- Software. Developed on Node 22.19, npm 10.9.3 and PostgreSQL 18.6 on Windows 11, and tested in desktop Chrome and iPhone Safari. The host, Render, runs the same code on Linux with a Neon database, which shows it is not tied to one operating system.
- Hardware. The pages are server-rendered with compressed responses and a minified Tailwind file, so they work on cheap phones and slow data. The layout was checked at phone, tablet and desktop widths. I have not run formal frame-rate or low-end device benchmarks, and I did not test on very old Android versions.
- Resource use. Distance and scoring are calculated in memory for a small list of upcoming events, and address lookups are cached in the database, so no page waits on an outside service.

3. Code quality and technical implementation

- Structure. Folders follow the job: `routes` (web requests), `services` (business rules), `models` (database), `middleware` (login checks, CSRF, rate limits), `sockets` (live features), `migrations`, `scripts` and `config`. No file does everything. The biggest file, the admin routes, is 819 lines of short handlers.
- Separation of concerns and SOLID. Each service has one job (`capacity.js` counts places, `identity.js` handles the NIN, `geo.js` does distance). The scoring engine is pure functions with its settings passed in, so it can be changed without touching the routes (open for change, closed for editing). Routes depend on services, not on each other.
- OOP. Sequelize models are classes. They carry their own rules: validation, constants such as the list of event categories, and hooks (`geoHooks.js` fills in coordinates when an address changes), which keeps that logic in one place. Beyond the models, I used modules rather than deep class hierarchies, because the logic here is mostly rules and data, and plain modules were easier to test. Notifications work in a polymorphic way: `notify` takes a message key and looks up its text in the language files, so a new kind of message is added with text only and the sender code does not change.
- Algorithms. Haversine distance, weighted scoring with a rolling diversity window, a locked transaction for safe booking, and HKDF key derivation with authenticated encryption. Each has tests.
- Readability. Names describe the job, comments explain the reason rather than repeat the code, the database changes are kept as 26 ordered migrations, and the README, `HANDOVER.md` and `docs/` explain how to run and hand over the project.
- Honest limits. Some route files are still long, and a few views repeat markup that could become partials. I would clean those next.